C03 ASSESSMENT TOOL 2

1. FIR Quantization using Truncation
Simulate a 16-tap FIR low-pass filter with cutoff frequency 0.4π rad/sample using 8-bit
fixed-point coefficient truncation and analyze the resulting quantization error.
(25) (BL3)
(WK4)
Given Data:
FIR Filter Length = 16 taps
Cutoff Frequency = 0.4π rad/sample
Coefficient Word Length = 8 bits
Input Signal = Sum of 500 Hz and 2 kHz sinusoids
Sampling Frequency = 8 kHz
Tasks:
1. Design the ideal FIR low-pass filter.
2. Quantize the coefficients using truncation.
3. Simulate the filter response using MATLAB/Python.
4. Plot the magnitude responses before and after quantization.
5. Determine the output error signal.
6. Calculate Mean Square Error (MSE).

SOLUTION :

Introduction

A Finite Impulse Response (FIR) filter is a digital filter widely used in signal processing because of its linear phase response, stability, and ease of implementation. In practical digital systems, filter coefficients cannot be stored with infinite precision and must be represented using a limited number of bits. This process is known as coefficient quantization. Quantization introduces small errors in the filter coefficients, which can affect the filter's frequency response and output signal.

In this experiment, a 16-tap FIR low-pass filter with a cutoff frequency of 0.4π rad/sample is designed. The filter coefficients are quantized using 8-bit fixed-point truncation, and the filter is simulated using MATLAB/Python. The original and quantized magnitude responses are compared, the output error signal is analyzed, and the Mean Square Error (MSE) is calculated to evaluate the effect of coefficient quantization on the filter performance.

Program

```
clc;
clear;
close all;

%% Given Data
N = 16;                % FIR Filter Length
Fc = 0.4;              % Normalized Cutoff Frequency (0.4*pi rad/sample)
Fs = 8000;             % Sampling Frequency
B = 8;                 % Coefficient Word Length

%% Step 1: Design FIR Low-Pass Filter
h = fir1(N-1, Fc, hamming(N));

%% Step 2: 8-bit Fixed-Point Coefficient Truncation
scale = 2^(B-1);       % 128
hq = fix(h * scale) / scale;

%% Display Coefficients
disp('Original Coefficients:');
disp(h);

disp('Quantized Coefficients:');
disp(hq);

%% Step 3: Input Signal (500 Hz + 2 kHz)
t = 0:1/Fs:0.02;       % 20 ms duration
x = sin(2*pi*500*t) + sin(2*pi*2000*t);
```

```
%% Filter Output
y_original = filter(h, 1,x);
y_quantized = filter(hq, 1,x);

%% Step 4: Frequency Response
[H,w] = freqz(h,1,1024);
[Hq,~] = freqz(hq,1,1024);

figure;

subplot(2,1,1);
plot(w/pi,20*log10(abs(H)),'b','LineWidth',1.5);
grid on;
title('Original FIR Filter Magnitude Response');
xlabel('Normalized Frequency (\times\pi rad/sample)');
ylabel('Magnitude (dB)');

subplot(2,1,2);
plot(w/pi,20*log10(abs(Hq)),'r','LineWidth',1.5);
grid on;
title('Quantized FIR Filter Magnitude Response');
xlabel('Normalized Frequency (\times\pi rad/sample)');
ylabel('Magnitude (dB)');

%% Step 5: Output Error Signal
error_signal = y_original - y_quantized;
```

```
figure;
plot(t,error _signal,'k','LineWidth',1.5);
grid on;
title('Output Error Signal');
xlabel('Time (seconds)');
ylabel('Amplitude');

%% Step 6: Mean Square Error (MSE)
MSE = mean(error_ signal.^2);

fprintf('\nMean Square Error (MSE) = %e\n', MSE);
```

Tool Used

- **Software:** MATLAB R2023a (or MATLAB R2022b/R2021a – any recent version)
- **Toolbox:** Signal Processing Toolbox
- **Programming Language:** MATLAB
- **Simulation Tool:** MATLAB Editor and Command Window

Result:

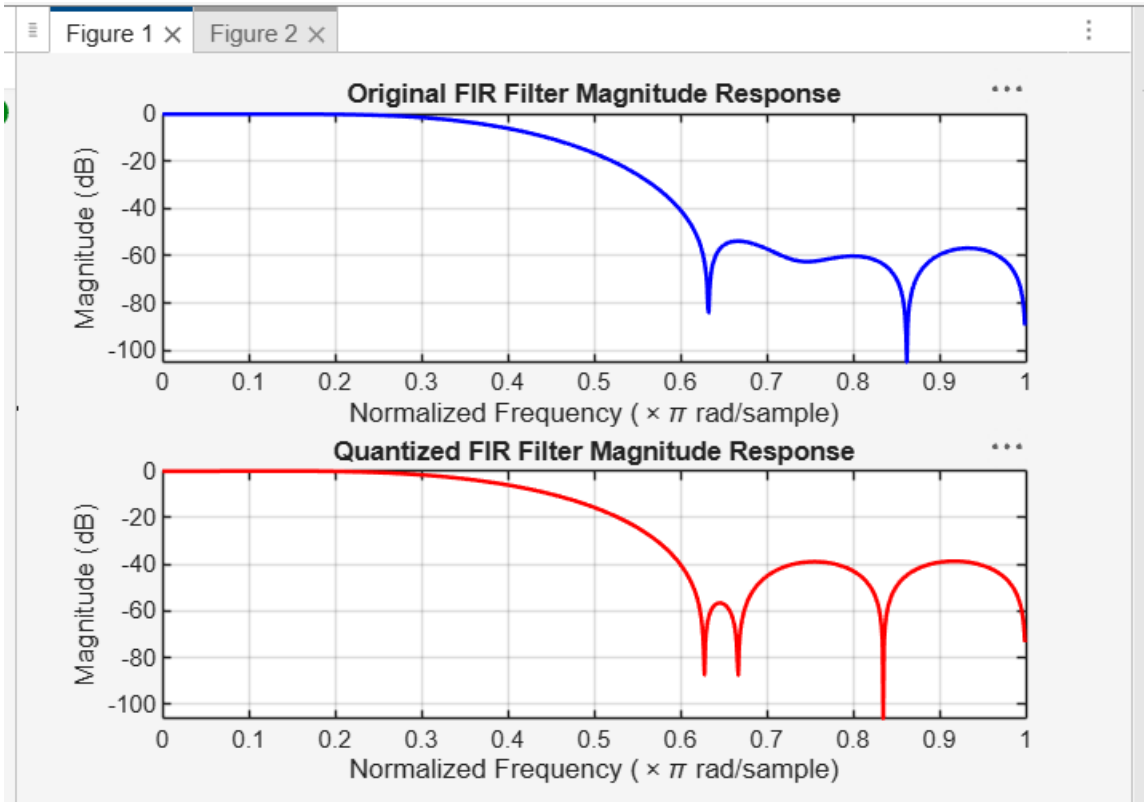
Figure 1
Figure 2
Original FIR Filter Magnitude Response
Magnitude (dB)
0
-20
-40
-60
-80
-100
0
0.1
0.2
0.3
0.4
0.5
0.6
0.7
0.8
0.9
1
Normalized Frequency (× π rad/sample)
Quantized FIR Filter Magnitude Response
Magnitude (dB)
0
-20
-40
-60
-80
-100
0
0.1
0.2
0.3
0.4
0.5
0.6
0.7
0.8
0.9
1
Normalized Frequency (× π rad/sample)

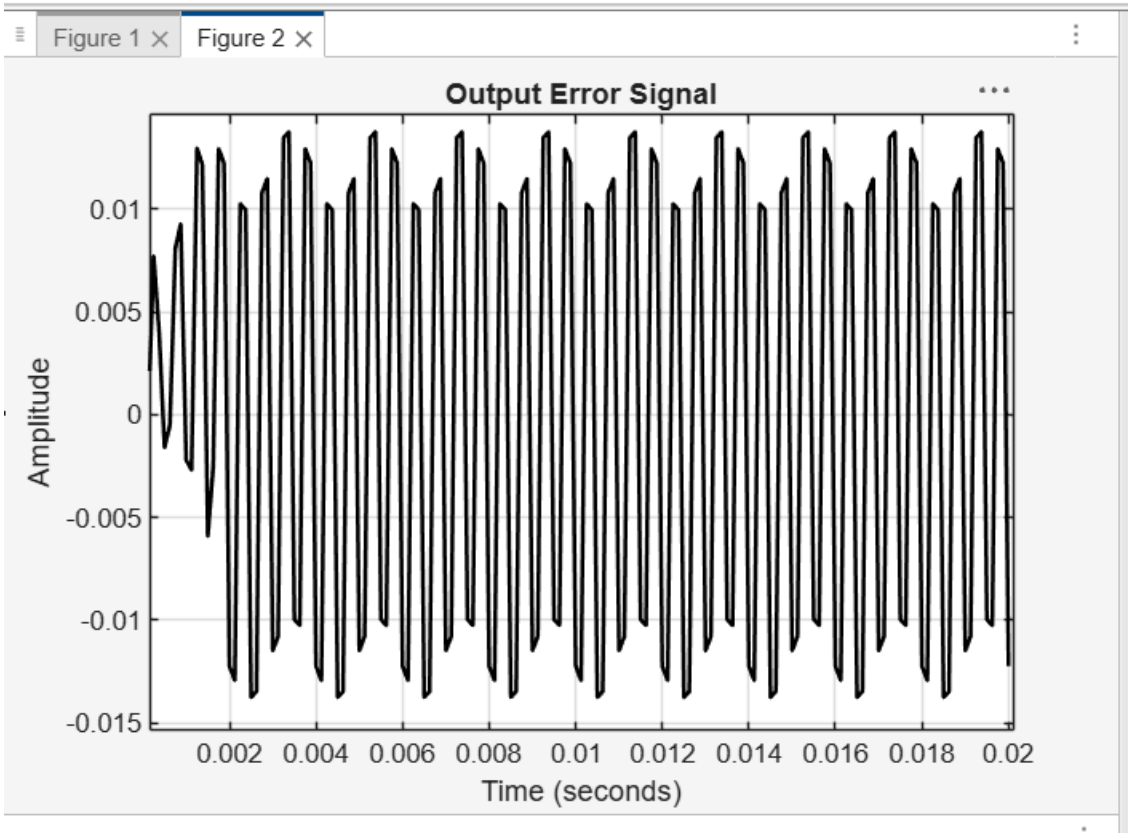
Figure 1
Figure 2
Output Error Signal
Amplitude
0.01
0.005
0
-0.005
-0.01
-0.015
0.002
0.004
0.006
0.008
0.01
0.012
0.014
0.016
0.018
0.02
Time (seconds)